

Hospital Automated Booking System

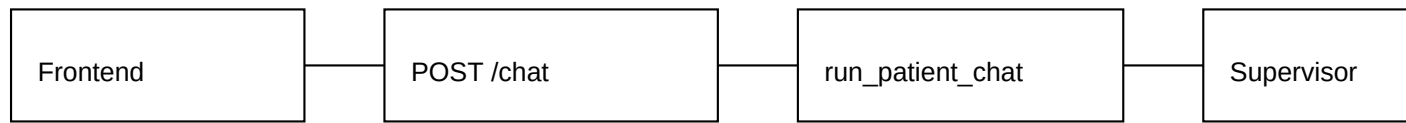
Project Architecture and Agent Flow

This PDF summarizes the current backend architecture, the LangGraph routing loop, and the main patient flows in the Smart Hospital Portal.

Core Architecture

- Frontend: `app/api/static/index.html`, `app.js`, and `styles.css` render the portal UI and send chat requests.
- API layer: `app/api/main.py` exposes `auth`, `chat`, `appointments`, and `admin` routers.
- Chat route: `app/api/routes/chat.py` loads the patient profile, history, active appointments, and executes the agent graph.
- Agent graph: `app/agents/graph.py` runs the LangGraph workflow and keeps the conversation memory compacted.
- Supervisor router: `app/agents/supervisor.py` decides the next specialist or ends the turn.
- Specialist agents: `triage_router`, `conversation_agent`, `remedy_agent`, `medical_rag`, and `appointment_booker` handle the medical and booking steps.
- Services: `app/services` and `app/db` provide chat history, appointment operations, memory policy, embeddings, vector search, token tracking, and persistence.

Primary Runtime Flow



The supervisor first checks deterministic shortcuts such as profile questions, booking lookup, cancellation, end-chat, and direct booking. If none match, it falls back to the LLM router. After routing, the selected node updates GraphState and returns control to the supervisor until a final_response is produced or the graph reaches finish.

Special Routes

- Profile query -> finish with a direct response from patient_profile
- Symptoms -> triage_router -> conversation_agent -> remedy_agent
- Persisting symptoms -> medical_rag -> appointment_booker
- Doctor/department request -> appointment_booker, or medical_rag first when the department is unknown
- Booking lookup / cancellation / reschedule -> appointment_booker
- End chat -> end_confirmation -> finish

Agents and Responsibilities

- Supervisor: decides whether the message continues the current flow or diverts to a specialist.
- Triage router: extracts intent, symptoms, and severity from the latest message.
- Conversation agent: asks follow-up intake questions until enough context is collected.
- Remedy agent: generates personalized care tips and handles follow-up about improvement or persistence.
- Medical RAG: maps symptoms and context to the most relevant department using heuristics, vector search, and LLM reasoning.

Appointment booker: handles doctor selection, slot selection, booking confirmation, rescheduling, and cancellation.

State That Drives Routing
awaiting, intent, symptoms, severity, target_department, remedy_given, persisting, booking_active, confirmed_booking, confirmed_bookings, chat_closed, and patient_profile.

Important Bypass Scenarios

- Chat already closed: `run_patient_chat` returns a closed-chat response immediately.
- User asks for profile info: the supervisor answers from `patient_profile` and skips the medical flow.
- User asks for a booking lookup: the supervisor returns the active or past bookings summary.
- User asks for a remedy during booking: the booking state is cleared and the flow returns to `remedy_agent`.
- RAG cannot confidently match a department: the flow returns to `conversation_agent` for more intake.
- Emergency severity: the remedy step is bypassed in favor of emergency guidance.

Data Stores and Supporting Services

- Chat history and session summaries are persisted by `app/services/chat_history.py` and `app/services/llm_usage.py`.
- Appointments use `app/services/appointments.py` and the routes in `app/api/routes/appointments.py`.
- Medical department matching uses `app/services/rag.py` with vector search and reranking.
- Database schema and rebuild scripts live under `app/db`.